DeHacker

Code Security Assessment

Butter Network (ButterSwap)

May 26th, 2026

Contents

Summary

DeHacker's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best prac tices. Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow/underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service/logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Issue Categories

Every issue in this report was assigned a severity level from the following:

Critical severity issues

A vulnerability that can disrupt the contract functioning in a number of scenarios or creates a risk that the contract may be broken.

Major severity issues

A vulnerability that affects the desired outcome when using a contract or provides the opportunity to use a contract in an unintended way.

Medium severity issues

A vulnerability that could affect the desired outcome of executing the contract in a specific scenario.

Minor severity issues

A vulnerability that does not have a significant impact on possible scenarios for the use of the contract and is probably subjective.

Informational

A vulnerability that has informational character but is not affecting any of the code.

Overview

Project Summary

Project Name	Butter Network (ButterSwap)
Platform	Butter Network
Website	https://www.butterswap.io/
Type	Dex
Language	Solidity
Codebase	https://github.com/butternetwork/butter-mos-contracts/tree/master/evmv3

Vulnerability Summary

Vulnerability Level	Total	Mitigated	Declined	Acknowledged	Partially Resolved	Resolved
Critical	0	0	0	0	0	0
Major	2	0	0	0	0	2
Medium	1	0	0	0	0	1
Minor	2	0	0	0	0	2
Informational	0	0	0	0	0	0
Discussion	0	0	0	0	0	0

Audit scope

ID	File	SHA256 Checksum
GLOBAL	https://github.com/butternetwork/butter-mos-contracts/tree/master/evmv3	7ed36ed60befa28dadd9d2101fe6178cb2f2addbf483792ba66b63e5541acbdc

Findings

ID	Title	Severity	Status
GLOBAL-01	Centralization Related Risks	Major	Resolved
GLOBAL-02	Centralization Exposure of Upgradeability	Major	Resolved
GLOBAL-03	ProtocolFee Violates CEI and Lacks nonReentrant Guard	Medium	Resolved
GLOBAL-04	Off-by-One Array Bound in NearDecoder.decodeNearSwapLog	Minor	Resolved
GLOBAL-05	VaultTokenV3.updateVault / transferToken: Vault Total Modified Without Emitting Events	Minor	Resolved

MAJOR

GLOBAL-01 | Centralization Related Risks

Issue	Severity	Status
Centralization Related Risks	Major	Resolved

Description

Butter MOS V3 is composed of multiple contracts. Each one exposes a large set of management functions, protected either by AuthorityManager (built on OZ AccessManager) or AccessControlEnumerable. **None of these functions are timelocked**; the permission boundary depends entirely on the off-chain security of the private keys / multisig signers. Privileged functions are enumerated below per contract.

AuthorityManager — central authorization contract, an OZ AccessManager . Custom role IDs include:

- ADMIN_ROLE = 1 — can call grantRole, revokeRole, setTargetFunctionRole, setTargetClosed , updateAuthority , etc. **Equivalent to the root permission of the entire protocol.**
- MANAGER_ROLE = 2 — operational role
- MINTER_ROLE = 10 — minter role
- Built-in ROOT_ROLE = 0 — the highest role of OZ AccessManager

Bridge.sol / BridgeAndRelay.sol / BridgeAbstract.sol — the bridge entities. Every restricted function below is controlled by AuthorityManager :

- trigger() — global pause / unpause
- registerTokenChains(token, toChains, enable) — declare which target chains a given token can be bridged to

- updateTokens(tokens, feature) — set token feature bits (including the MINTABLE_TOKEN bit, which decides whether the token follows the mint / burn path)
- setTrustAddress(addr, enable) — maintain the cross-chain trusted-sender allowlist (affects how initiator is resolved)
- setServiceContract(t, addr) — replace any of the referenced contracts: wToken , lightNode , feeService , swapLimit , tokenRegister
- setFailedReceiver(addr) — set the fallback receiver. When inbound transfers fail, funds are routed to this address.
- setRelay(chainId, relay) (Bridge only) — set the relay chain ID and relay MOS address
- setDistributeRate(id, to, rate) (BridgeAndRelay only) — set the vault / relayer / protocol fee distribution rates, which directly determine how each cross-chain fee is divided
- registerChain(chainIds, addresses, type) (BridgeAndRelay only) — register external chain MOS addresses and chain types (EVM / NEAR / TON / Solana / BTC / XRP)
- _authorizeUpgrade(newImpl) — **UUPS upgrade entry point** (see Global #2 for details)

Recommendation

This finding describes the level of decentralization of the project. It is recommended to strengthen security from the following angles:

- **Privileged roles must be held by multisig wallets**: AuthorityManager.ADMIN_ROLE must be controlled by a multisig (e.g. Gnosis Safe). **EOAs are strictly forbidden**. The current deployment script in tasks/subs/authority.js defaults admin to the deployer EOA (if (admin === "") admin = deployer.address;). **Before any mainnet deployment, a multisig address must be passed in explicitly.**
- Critical operations should go through a timelock, giving off-chain monitoring a response window. Highest-priority items:
  - _authorizeUpgrade (see Global #2)
  - setServiceContract (especially lightNode)
  - updateTokens (the MINTABLE bit is the on/off switch for the mint capability)
  - setDistributeRate (fee distribution)
- **The bridge-side control of MINTABLE tokens must be audited together with the token's own minter model**. If a Butter MOS bridge contract is simultaneously an unrestricted minter on some ERC20, the supply security of that token is equivalent to the security of the bridge. Both must be audited with the same depth and protected with the same strength. It is strongly recommended that the token-side minter be hardened (multisig and / or rate-limited mint), and that the bridge-side and token-side interface be audited independently

MAJOR

GLOBAL-02 | Centralization Exposure of Upgradeability

Issue	Severity	Status
Centralization Exposure of Upgradeability (UUPS Proxy Pattern)	Major	Resolved

Description

This project relies heavily on OpenZeppelin's **UUPS (EIP-1822) + ERC1967 proxy** pattern. Most of the core contracts can be replaced via upgrade. This means that even if all non-admin entry pointsare secure, **anyone holding the upgrade key can replace the entire bridge logic with arbitrary code**.

Proxy shell:

- **OmniServiceProxy.sol** — a minimal wrapper around ERC1967Proxy . All UUPS contracts are deployed behind their own instance of OmniServiceProxy .

Upgradeable implementation contracts (5 in total)

Implementation	Upgrade authority	Upgrade entry
Bridge.sol	`AuthorityManager.restricted`	`upgradeToAndCall(newImpl, data)`
BridgeAndRelay.sol	`AuthorityManager.restricted`	same
TokenRegisterV3.sol	`UPGRADER_ROLE` (independent AccessControl)	same
ProtocolFee.sol	`AuthorityManager.restricted`	same
DepositWhitelist.sol	`AuthorityManager.restricted`	same

Non-upgradeable contracts (plain contracts; the deployed bytecode is final): VaultTokenV3 , FeeService , AuthorityManager , OmniServiceProxy , and the quoter / validator helpers under lens/ .

Recommendation

- **Force the upgrade authority to be held by a multisig**. Before any network goes live, it must be publicly verified that each of the following addresses is a multisig / timelock — **EOAs are strictly forbidden**:
  - The ADMIN_ROLE holder of AuthorityManager on every network
  - The UPGRADER_ROLE holder of TokenRegisterV3 on every network
  - The DEFAULT_ADMIN_ROLE holder of TokenRegisterV3 on every network (it can grantRole to escalate itself)
- **Gate critical upgrade paths behind a timelock**. _authorizeUpgrade on the bridge-class contracts must not be triggerable by a single signature; it must go through a public waiting window of ≥ 2 days (commit / reveal), and emit an easily-monitorable UpgradeProposed event.
- **Reserve storage with __gap arrays.** Add uint256[50] private __gap; to BridgeAbstract, TokenRegisterV3, and periphery/base/BaseImplementation. **This must be done before thenext upgrade**, otherwise any new state field added in a future OZ parent-contract version will cause a storage collision on upgrade.

MEDIUM

GLOBAL-03 | ProtocolFee Violates CEI and Lacks nonReentrant Guard

Issue	Severity	Status
ProtocolFee Violates CEI (checks-effects-interactions) and Has No nonReentrant Guards	Medium	Resolved

Description

ProtocolFee.sol violates the checks-effects-interactions pattern in two places, and the contract as a whole does not inherit ReentrancyGuardUpgradeable — no function carries a nonReentrant modifier.

First case — _keepAccountSingleToken (**ProtocolFee.sol:212-226**) makes the external call feeTreasury.withdrawFee(...) **before** accumulating accumulated[*] :

```
function _keepAccountSingleToken(address token) internal {
    uint256 unAccount = _getUnAccount(token);
    uint256 treasuryAmount;
    if(address(feeTreasury) != address(0)) {
        treasuryAmount = feeTreasury.feeList(address(this), token);
        if(treasuryAmount > 0) feeTreasury.withdrawFee(address(this), token);   // ❶ ex
    }
    uint256 total = treasuryAmount + unAccount;
    if(total > 0){
        accumulated[token][FeeType.DEV]     += _getShareAmount(FeeType.DEV, total);
        accumulated[token][FeeType.BUYBACK] += _getShareAmount(FeeType.BUYBACK, total);
        accumulated[token][FeeType.RESERVE] += _getShareAmount(FeeType.RESERVE, total);
        accumulated[token][FeeType.STAKER]  += _getShareAmount(FeeType.STAKER, total);
    }
}
```

Second case — claimWithTargetToken (**ProtocolFee.sol:161-181**) issues one external swap.swap(...) call **per token inside a loop**, with reads and writes of accumulated / claimed interleaved between them:

```
function claimWithTargetToken(FeeType feeType, address[] memory tokens, address targetT
    uint256 len = tokens.length;
    uint256 totalAmount;
    for (uint256 i = 0; i < len; i++) {
        address token = tokens[i];
        _keepAccountSingleToken(token);                                    // (a) con
        uint256 claimable = accumulated[token][feeType] - claimed[token][feeType];
        if(claimable > 0) {
            claimed[token][feeType] += claimable;                          // (b) sta
            if(token != targetToken) {
                SafeERC20.forceApprove(IERC20(token), address(swap), claimable);
                totalAmount += swap.swap(token, targetToken, claimable, 1, address(this
            } else {
                totalAmount += claimable;
            }
            emit ClaimFee(feeType, token, claimable);
        }
    }
    _release(targetToken, feeShares[feeType].receiver, totalAmount);
}
```

The shared problem of both cases: **external calls precede state writes**. Any callback path triggered by feeTreasury / swap can observe an inconsistent intermediate state, leading to incorrect share allocation. In addition, **the contract does not inherit ReentrancyGuardUpgradeable and no function carries nonReentrant** — even if every external counterparty is trusted, this still violates the audit-industry baseline "external call + state write must be reentry-safe"

Recommendation

- **Reorder _keepAccountSingleToken to follow CEI** (smallest change, preferred): take a snapshot of the treasury balance up front, perform the local accumulated[*] updates, and only then call feeTreasury.withdrawFee :

```
function _keepAccountSingleToken(address token) internal {
    uint256 unAccount = _getUnAccount(token);
    uint256 treasuryAmount;
    if(address(feeTreasury) != address(0)) {
        treasuryAmount = feeTreasury.feeList(address(this), token);
    }
    uint256 total = treasuryAmount + unAccount;
    if(total > 0){
        accumulated[token][FeeType.DEV]     += _getShareAmount(FeeType.DEV, total);
        accumulated[token][FeeType.BUYBACK] += _getShareAmount(FeeType.BUYBACK, tot
        accumulated[token][FeeType.RESERVE] += _getShareAmount(FeeType.RESERVE, tot
        accumulated[token][FeeType.STAKER]  += _getShareAmount(FeeType.STAKER, tota
    }
    if(treasuryAmount > 0) feeTreasury.withdrawFee(address(this), token);
}
```

- **Make ProtocolFee inherit ReentrancyGuardUpgradeable**, call__ReentrancyGuard_init() in initialize, and add nonReentrant to claim / claimWithTargetToken / updateShares. updateShares internally invokes _keepAccount → _keepAccountSingleToken , so it must be guarded as well.
- **Move the swap call out of the loop in claimWithTargetToken** : first accumulate the claimed updates across the loop, then issue a single consolidated swap after the loop ends; alternatively, run each per-token swap behind a strict reentry-safe wrapper (single call + nonReentrant).

MINOR

GLOBAL-04 | Off-by-One Array Bound in NearDecoder.decodeNearSwapLog

Issue	Severity	Status
Off-by-One Array Bound in NearDecoder.decodeNearSwapLog	Minor	Resolved

Description

decodeNearSwapLog at **NearDecoder.sol:31-50** treats the RLP-decoded logList as a 10-field tuple, but the guard only checks length >= 9 :

```
function decodeNearSwapLog(bytes memory log) internal pure returns (MessageOutEvent mem
    bytes memory logByts = hexStrToBytes(log);
    RLPReader.RLPItem[] memory logList = logByts.toRlpItem().toList();
    if (logList.length < 9) revert logs_length_too_low();          // ← guard requires
    _outEvent = MessageOutEvent({
        ...
        swapData: logList[8].toBytes(),
        mos:      logList[9].toBytes(),                            // ← actually indexe
        gasLimit: 0
    });
}
```

The indices [0..9] use **10 elements**, but the guard requires only logList.length >= 9 . When logList.length == 9 :

- the guard 9 < 9 evaluates to false and **passes**;
- the subsequent access logList[9] ⇒ RLPReader out-of-bounds ⇒ revert.

This function is invoked by **BridgeAndRelay.messageIn** under the chainTypes[_chainId] == ChainType.NEAR branch, handling inbound cross-chain messages from NEAR. A revert here means inbound NEAR messages cannot be processed at all — user funds are locked on the NEAR side and an emergency upgrade of the EVM contract is required to release them.

Recommendation

- Minimal fix: tighten the guard to match the actual field count expected by the EVM side:
  if (logList.length < 10) revert logs_length_too_low()

MINOR

GLOBAL-05 | VaultTokenV3.updateVault / transferToken: Vault Total Modified Without Emitting Events

Issue	Severity	Status
VaultTokenV3.updateVault / transferToken Modify the Vault Total Without Emitting Events	Minor	Resolved

Description

VaultTokenV3.sol contains two functions that write to totalVault and vaultBalance[chain] , but **neither emits any event:**

VaultTokenV3.sol:151-170 — updateVault has its emit left as a TODO comment:

```
event UpdateVault(address indexed token, uint256 fromChain, uint256 toChain, uint256);
...
function updateVault(
    uint256 _fromChain,
    uint256 _amount,
    uint256 _toChain,
    uint256 _outAmount,
    uint256 _relayChain,
    uint256 _vaultFee
) external override onlyManager {
    vaultBalance[_fromChain] += _amount.toInt256();
    vaultBalance[_toChain]   -= _outAmount.toInt256();
    totalVault += _vaultFee;                          // totalVault is mutated
    _chainSet.add(_fromChain);
    _chainSet.add(_toChain);
    _chainSet.add(_relayChain);
    // todo
    // emit UpdateVault();                             // ← will never fire
}
```

VaultTokenV3.sol:132-149 — transferToken also mutates totalVault , but **does not even declare an event**:

```
function transferToken(
    uint256 _fromChain,
    uint256 _amount,
    uint256 _toChain,
    uint256 _outAmount,
    uint256 _relayChain,
    uint256 _fee
) external override onlyManager {
    vaultBalance[_fromChain] += _amount.toInt256();
    vaultBalance[_toChain]   -= _outAmount.toInt256();
    uint256 vaultFee = _amount - _outAmount - _fee;
    totalVault += vaultFee;                          // totalVault is mutated here as
    _chainSet.add(_fromChain);
    _chainSet.add(_toChain);
    _chainSet.add(_relayChain);
}
```

VaultTokenV3 is an LP token whose exchange rate is determined by totalVault / totalSupply() . Both updateVault and transferToken mutate both numerator and denominator, but the missing events imply:

- **Front-ends can only poll totalVault / totalSupply to infer exchange-rate changes**, with no near-real-time event subscription;
- **Off-chain indexers cannot reconstruct the LP exchange-rate history from event streams alone** — they have to snapshot periodically and will lose intermediate states;
- **Auditing / reconciliation becomes harder** — the same contract's DepositVault and WithdrawVault properly emit events, so the gap here looks like a missing implementation rather than an intentional design, and reviewers may incorrectly conclude that no global accounting has changed.

This is not exploitable, but it pollutes downstream integration.

Recommendation

- Add events for both updateVault and transferToken . Including the previous total vault and block.timestamp is recommended, so that off-chain consumers can directly distinguish a "routine cross-chain vault update" from an abnormal jump:

```
event UpdateVault(
    address indexed token,
    uint256 fromChain,
    uint256 toChain,
    uint256 relayChain,
    uint256 amount,
    uint256 outAmount,
    uint256 vaultFee,
    uint256 previousTotalVault,
    uint256 newTotalVault
);

event TransferTokenVault(
    address indexed token,
    uint256 fromChain,
    uint256 toChain,
    uint256 relayChain,
    uint256 amount,
    uint256 outAmount,
    uint256 fee,
    uint256 vaultFee,
    uint256 previousTotalVault,
    uint256 newTotalVault
);
```

- At the same time, remove the // todo comment so future readers don't mistake this for "still under debugging".

Disclaimer

This report is based on the scope of materials and documentation provided for a limited review at the time provided. Results may not be complete nor inclusive of all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. A report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on the reports in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, we disclaim all warranties, expressed or implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. We do not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Appendix

Finding Categories

Centralization / Privilege
Centralization / Privilege findings refer to either feature logic or implementation of components that act against the nature of decentralization, such as explicit ownership or specialized access roles in combination with a mechanism to relocate funds.

Coding Style
Coding Style findings usually do not affect the generated bytecode but rather comment on how to make the codebase more legible and, as a result, easily maintainable.

Volatile Code
Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases that may result in a vulnerability.

Logical Issue
Logical Issue findings detail a fault in the logic of the linked code, such as an incorrect notion on how block. timestamp works.

Checksum Calculation Method

The "Checksum" field in the "Audit Scope" section is calculated as the SHA-256 (Secure Hash Algorithm 2 with digest size of 256 bits) digest of the content of each file hosted in the listed source repository under the specified commit.

The result is hexadecimal encoded and is the same as the output of the Linux “sha256sum" command against the target file.

About

DeHacker is a team of auditors and white hat hackers who perform security audits and assessments. With decades of experience in security and distributed systems, our experts focus on the ins and outs of system security. Our services follow clear and prudent industry standards. Whether it's reviewing the smallest modifications or a new platform, we'll provide an in-depth security survey at every stage of your company's project. We provide comprehensive vulnerability reports and identify structural inefficiencies in smart contract code, combining high-end security research with a real-world attacker mindset to reduce risk and harden code.

BLOCKCHAIINS

 Ethereum

 Cosmos

 Eos

 Substrate

TECH STACK

 Python

 Solidity

 Rust

 c++

CONTACTS

 https://dehacker.io

 https://twitter.com/dehackerio

 https://github.com/dehacker/audits_public

 https://t.me/dehackerio

 https://blog.dehacker.io/

DeHacker

May 26th 2026